

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 5: Arrays

Learning Outcomes

- What are arrays and how use them in Java?
- How to loop through arrays in Java?
- What are multidimensional arrays?

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

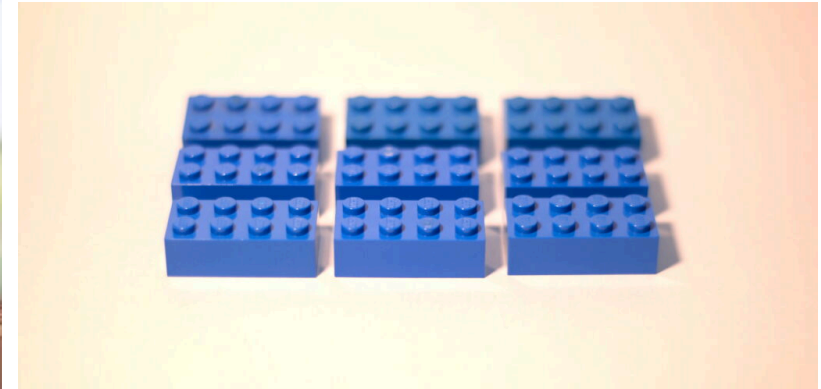
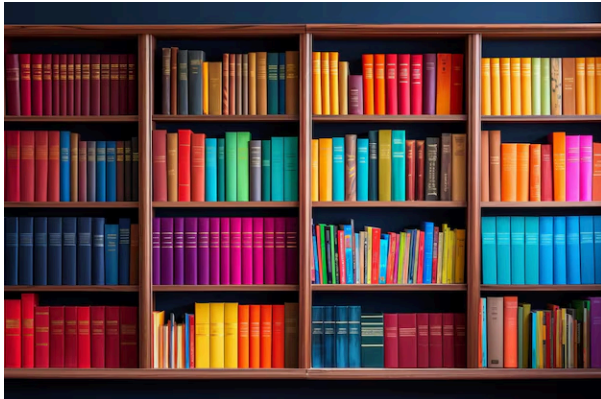
Some of the instructions may be **repeated in loops**.

What is a program?

A sequence of instructions that is **executed** by a computer.

Some of the instructions may be **repeated** in loops.

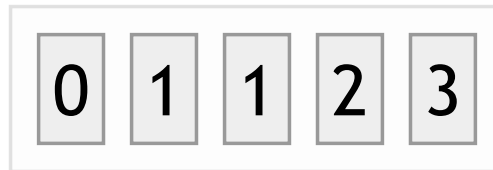
Data is usually **organized in linear structure** called **array** to be processed in a loop.



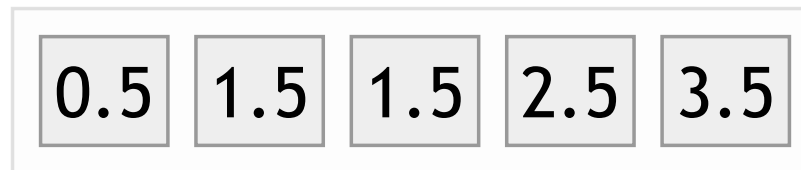
Introduction to Array

- Array is a collection of elements of the same type stored in contiguous memory.
- Arrays have a fixed size that must be specified at initialization.

int Array example with first 5 Fibonacci numbers



double Array example



Introduction to Array

String Array example with 5 strings



boolean Array example with 5 boolean values



Introduction to Array

Keep in mind that an array can be of any type

- all elements from the same type
- But not mixed types !
 - ❑ Wrong example of array mixing types



Declaring an Array in Java

Syntax

```
TYPE[] VAR_NAME;
```

Example int array

```
int[] arr;
```

Example String array

```
String[] arr;
```

Allocating an Array in Java

Keep in mind that **declaring an array does not allocate its memory**

- To allocate memory for the array in Java you need to use the keyword `new`

Declaring and allocating

Syntax

```
TYPE[] VAR_NAME = new TYPE[SIZE];
```

Example int array

```
int[] arr = int[5];
```

Example String array

```
String[] arr = new String[5];
```

Allocating an Array in Java

Example of declaring WITHOUT allocating

```
int[] arr; // null
```

Example of declaring and allocating

```
int[] arr = new int[5] // [0, 0, 0, 0, 0 ]
```

Allocating a Array in Java

Once allocated, an array size cannot be changed!

- But you can allocate again with a new size!

```
int[] arr = new int[5] // [0, 0, 0, 0, 0 ]
```

```
arr = new int[10] // [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Allocating a Array in Java

- Usually, you know size needed
- But sometime the size is only known when user runs program and defines somehow
 - One strategy is to think – what is max size needed!

```
int[] arr = new int[100]
```

- Other approach is use dynamic structure like `ArrayList` (discussed in Week 7)

Array Indexing

Array Indexing

- Arrays in Java have **zero-based indexing**
 - the first element is at index `0`
 - the last element is at `array length - 1`
- **Most of programming languages are zero-based indexing**, such as Python, Rust and C.
 - **Just few index elements by `1`**, such as Matlab and Lua.

Array Indexing

Array `arr` with numbers from 0 to 4

i	0	1	2	3	4
arr[i]	0	1	2	3	4

```
arr[1] = "42";
```

i	0	1	2	3	4
arr[i]	0	42	2	3	4

Array Indexing

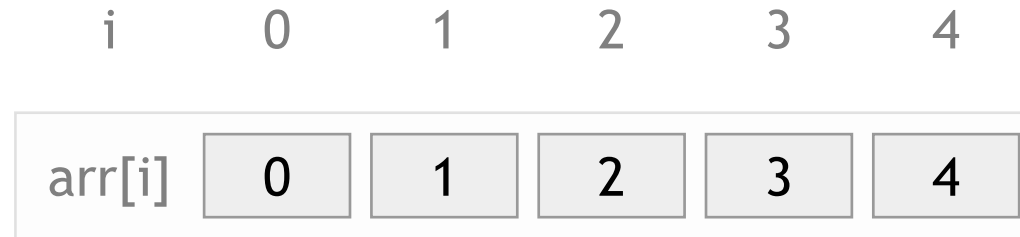
Array `arr` with 5 strings

i	0	1	2	3	4
arr[i]	Imperative	Programming	is	nice	!

```
arr[3] = "amazing";
```

i	0	1	2	3	4
arr[i]	Imperative	Programming	is	amazing	!

Indexing error



A error **IndexOutOfBoundsException** may occurs when trying to access an index outside the valid range.

```
System.out.println(arr[0]); // Accessing the first element
System.out.println(arr[5]); // Error when accessing the sixth element
```

The access of memory out the program bounds is severe security issue. We discuss about it at **Week 12** .

Array as param and return of a function

This function increment by one all values of an array given as param

```
int[] incrementArray(int[] numbers) {  
    int[] incrementedArray = new int[numbers.length];  
    for (int i = 0; i < numbers.length; i++) {  
        incrementedArray[i] = numbers[i] + 1;  
    }  
    return incrementedArray;  
}
```

Initializing an Array

All above array examples were already Initialised to help understand the indexing

- But when programming in Java, you usually need to initialize the array before use it

Initialising a int array </>

```
int[] arr = new int[5];    // Declaration and allocating
arr[0] = 1;                // Initializing first element
arr[1] = 2;                // Initializing second element
arr[2] = 3;                // Initializing third element
arr[3] = 4;                // Initializing forth element
arr[4] = 5;                // Initializing fifth element
```

Shortcut

```
int[] arr = {1, 2, 3, 4, 5};
```

Initialising a String array </>

```
String[] arr = new String[5];           // Declaration and allocating
arr[0] = "Imperative";                  // Initializing first element
arr[1] = "Programming";                 // Initializing second element
arr[2] = "is";                          // Initializing third element
arr[3] = "nice";                        // Initializing fourth element
arr[4] = "!";                          // Initializing fifth element
```

Shortcut

```
String[] arr = {"Imperative", "Programming", "is", "nice", "!"};
```

Default initialisation when allocating

Keep mind that in Java does a default initialisation when allocating basic types such as `int`, `double` and `boolean`

```
int[] arr = new int[5]; // [ 0, 0, 0, 0, 0 ]
```

```
double[] arr = new double[5]; //[ 0.0, 0.0, 0.0, 0.0, 0.0 ]
```

```
int boolean = new boolean[5]; // [ false, false, false, false, false ]
```

Default initialisation when allocating

But Java **DOES NOT DO** a default initialisation of `String` type

```
String[] arrString = new String[5]  
System.out.println(Arrays.toString(arrString));
```

```
[ null, null, null, null, null ]
```

```
String[] arrString = {"a", "b", "c"}  
System.out.println(Arrays.toString(arrString));
```

```
[a, b, c]
```

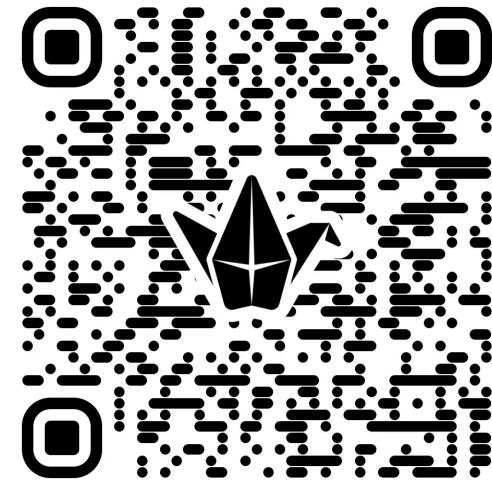
- The default `null` It happens because `String` is a Java `class` . But you will learn more about it in `CS10P - Object-Oriented Programming`

Quiz 1 ([click here](#) or scan QRCode)

Which is the output of the following code?

```
int[] arr = {1, "two", 3};  
System.out.println(arr[1]);
```

- a. two
- b. 1
- c. 3
- d. Compile error



Loop through arrays

Looping Through Arrays

Data is usually organized in linear structure called array to be processed in a loop.

- You can also say loops iterate through arrays.

Looping Through Arrays

To iterate through, you need remember that Java have **zero-based indexing**

- You must iterate for `0` to `array length`
- Every has array has property called `length` to access the allocated size of the array.
 - You can access it by `ARRAY_NAME.length`

array `length`

```
int[] arr = {1, 2, 3, 4, 5};  
System.out.println(arr.length)  
System.out.println(Arrays.toString(arr))
```

```
5  
[ 1, 2, 3, 4, 5 ]
```

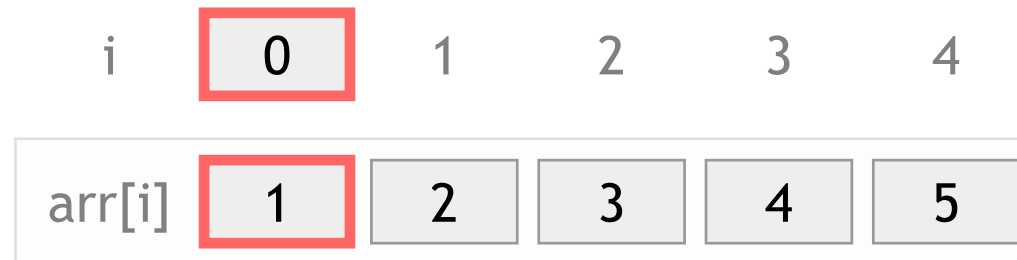
for iterating through an int array

```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

i	0	1	2	3	4
arr[i]	1	2	3	4	5

for iterating through an int array (at index 0)

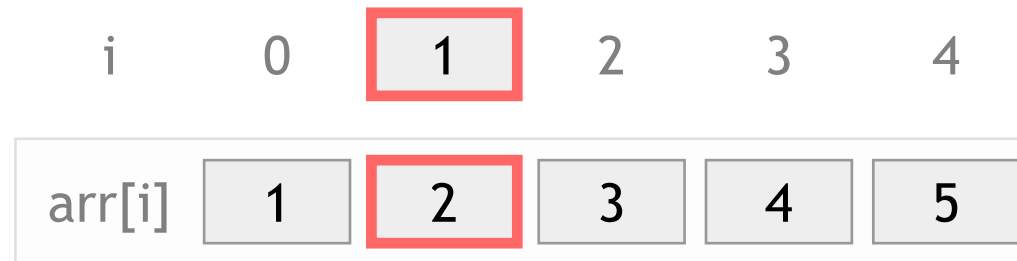
```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```



1

for iterating through an int array (at index 1)

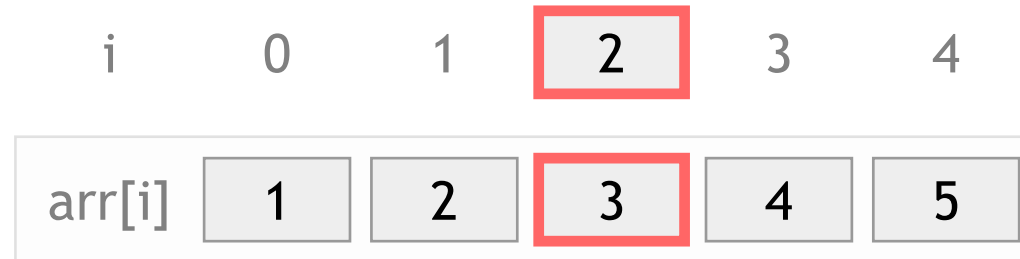
```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```



1
2

for iterating through an int array (at index 2)

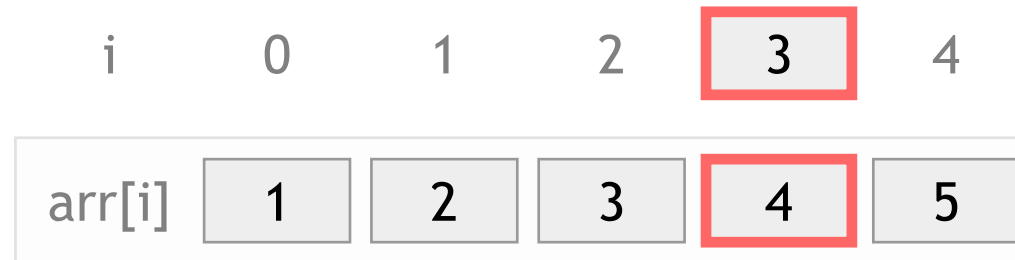
```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```



1
2
3

for iterating through an int array (at index 3)

```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```



1
2
3
4

for iterating through an int array (at index 4)

```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

i	0	1	2	3	4
arr[i]	1	2	3	4	5

1
2
3
4
5

for iterating through a String array

```
String[] arr = {"Imperative", "Programming", "is", "nice", "!"};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

i

0

1

2

3

4

arr[i]

Imperative

Programming

is

nice

!

```
Imperative  
Programming  
is  
nice  
!
```

array length vs String length()

To not misunderstood them

- Java array has length property
- Java String has length() function

```
int[] arr = {1, 2, 3, 4, 5};  
System.out.println(Arrays.toString(arr));  
System.out.println(arr.length);  
String str = new String("Imperative");  
System.out.println(str.length());
```

```
[ 1, 2, 3, 4, 5 ]  
5  
10
```

array `length` vs String `length()`

for iterating through an int array using `length`

```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

for iterating through String using `length()`

```
String str = new String("Imperative");  
for (int i = 0; i < str.length(); i++) {  
    System.out.println(str[i]);  
}
```

"For each" For Loop

To avoid manage the length you can also use the following syntax (also called "for each")

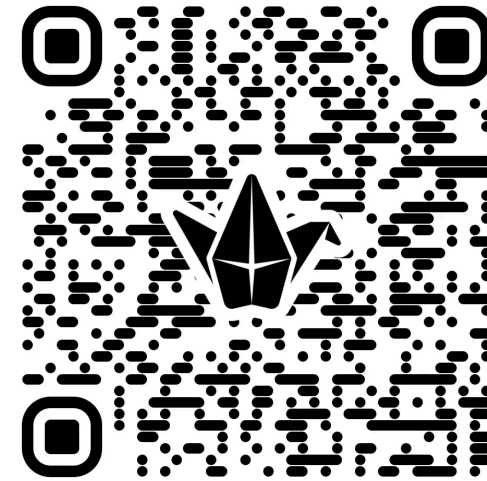
```
int[] arr = {1, 2, 3, 4, 5};  
for (int element : arr) {  
    System.out.println(element);  
}
```

Quiz 2 ([click here](#) or scan QRCode)

Which is the output of the following code?

```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length; i = i + 2) {  
    System.out.print(arr[i] + " ");  
}
```

- a. 1 2 3 4 5
- b. 1 3 5
- c. 1 2 4
- d. 1 3



Arrays.toString()

Easy alternative to use for loops to print a array. `Arrays.toString()` convert the array to a string.

```
String[] arr = {"a", "b", "c"}  
System.out.println(Arrays.toString(arr));
```

```
[a, b, c]
```

```
int[] arr = {1, 2, 3, 4, 5};  
System.out.println(Arrays.toString(arr));
```

```
[1, 2, 3, 4, 5]
```

Multidimensional Arrays

Multidimensional Arrays

- Multidimensional arrays are arrays of arrays.
- Commonly used to represent matrices.
- use the `[][]` syntax

Declaring and Initializing 2D Arrays </>

```
int[][] matrix = new int[3][3]; // 3x3 matrix
matrix[0][0] = 1;                // Initialize one value
```

Shortcut Initialization for 2D Arrays </>

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Accessing 2D Array Elements

- Accessing elements using row and column indices.

```
int value = matrix[0][1]; // Accessing element at row 0, column 1
System.out.println(value); // Output: 2
```

Looping Through 2D Arrays

```
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

Why should we use `print` instead of `println` inside of the inner loop?

IndexOutOfBoundsException in Multidimensional Arrays

- `IndexOutOfBoundsException` also applies to multidimensional arrays.

Example of Index Overflow

```
int[][] matrix = new int[2][2];  
matrix[2][1] = 5; // Causes IndexOutOfBoundsException
```

- This error occurs because we are trying to access a row beyond the valid index.

Using `for-each` with a Multidimensional Array `</>`

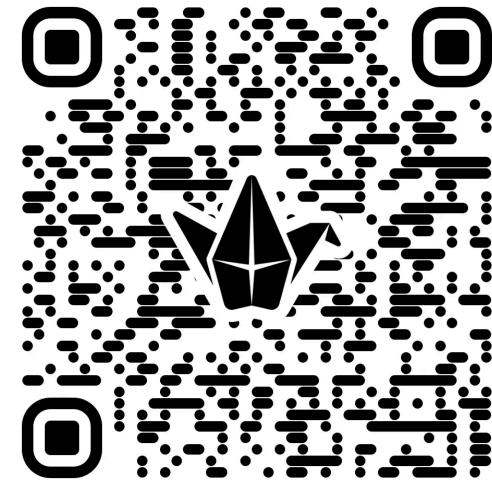
The `for-each` loop can also be used to iterate through a **multidimensional array**, accessing each sub-array and its elements in a nested manner.

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
for (int[] row : matrix) {  
    for (int element : row) {  
        System.out.print(element + " ");  
    }  
    System.out.println();  
}
```

Quiz 3 ([click here](#) or scan QRCode)

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
int sum = 0;  
for (int i = 0; i < matrix.length; i++) {  
    sum += matrix[i][i];  
}  
System.out.println(sum);
```

- a. 15
- b. 12
- c. 9
- d. 6



Matrix Operations using Multidimensional array

- Operations on matrices include checking for special properties, such as:
 - Identity matrix
 - Determinant
 - Transpose

Identity Matrix

- An **identity matrix** is a square matrix where all diagonal elements are 1 and all off-diagonal elements are 0.
- Example of a 3x3 identity matrix:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Identity Matrix

```
boolean isIdentityMatrix(int[][] matrix) {  
    for (int i = 0; i < matrix.length; i++) {  
        for (int j = 0; j < matrix[i].length; j++) {  
            if (i == j && matrix[i][j] != 1) {  
                return false;  
            } else if (i != j && matrix[i][j] != 0) {  
                return false;  
            }  
        }  
    }  
    return true;  
}
```

- The function checks the diagonal for 1s and all other elements for 0s.

Identity Matrix

```
boolean isIdentityMatrix(int[][] matrix) {  
    for (int i = 0; i < matrix.length; i++) {  
        for (int j = 0; j < matrix[i].length; j++) {  
            if (i == j && matrix[i][j] != 1) {  
                return false;  
            } else if (i != j && matrix[i][j] != 0) {  
                return false;  
            }  
        }  
    }  
    return true;  
}
```

```
int[][] matrix = {  
    {1, 0, 0},  
    {0, 1, 0},  
    {0, 0, 1}  
};
```

Identity Matrix

```
boolean isIdentityMatrix(int[][] matrix) {  
    for (int i = 0; i < matrix.length; i++) {  
        for (int j = 0; j < matrix[i].length; j++) {  
            if (i == j && matrix[i][j] != 1) {  
                return false;  
            } else if (i != j && matrix[i][j] != 0) {  
                return false;  
            }  
        }  
    }  
    return true;  
}
```

```
int[][] matrix = {  
    {1, 0, 0},  
    {0, 2, 0},  
    {0, 0, 1}  
};
```

Determinant of a 2x2 Matrix

- We can calculate the determinant of a 2x2 matrix by:

$$\det(A) = (a_{00} \times a_{11}) - (a_{01} \times a_{10})$$

Java code

```
int determinant2x2(int[][] matrix) {  
    return (matrix[0][0] * matrix[1][1]) - (matrix[0][1] * matrix[1][0]);  
}
```

Determinant of a 2x2 Matrix

```
int determinant2x2(int[][] matrix) {  
    return (matrix[0][0] * matrix[1][1]) - (matrix[0][1] * matrix[1][0]);  
}
```

```
int[][] matrix = {  
    {4, 3},  
    {6, 3}  
};  
int det = determinant2x2(matrix);  
System.out.println("Determinant: " + det); // Output: -6
```

Matrix Transpose

The transpose of a matrix A , denoted as A^T , is obtained by swapping its rows and columns.

Given a matrix:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

The transpose of A , written as A^T , is:

$$A^T = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ a_{13} & a_{23} \end{bmatrix}$$

Matrix Transpose

```
int[][] transpose(int[][] matrix) {  
    // allocate the new matrix  
    int[][] result = new int[matrix[0].length][matrix.length];  
  
    // transpose matrix to result  
    for (int i = 0; i < matrix.length; i++) {  
        for (int j = 0; j < matrix[i].length; j++) {  
            result[j][i] = matrix[i][j];  
        }  
    }  
    return result;  
}
```

- We need allocate a new matrix for the result
 - `matrix[0].length` is the length of the first row

Matrix Transpose </>

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6}  
};  
int[][] transposed = transpose(matrix);  
for (int i = 0; i < transposed.length; i++) {  
    for (int j = 0; j < transposed[i].length; j++) {  
        System.out.print(transposed[i][j] + " ");  
    }  
    System.out.println();  
}
```

Output:

```
1 4  
2 5  
3 6
```

Review Learning Outcomes

Best Practices with Arrays

- Think about the type to be used
 - avoid unnecessary type conversions and increase readability
- Initialize Arrays Properly
 - Always ensure arrays are initialised before using it
- Use length property
 - When iterating over arrays, use the `array.length` property to avoid accessing out-of-bounds indices.
 - Or use a `for-each` syntax
- Clear intent with Comments
 - When using multidimensional arrays or complex structures, comment you loop

Review Learning Outcomes

What are arrays and how use them in Java?

- **Array** is a collection of elements of the same type stored in contiguous memory.
- **Array** Allow manipulation of data in sequence.
- To use array, you may need **declare, allocate and initialize**.

How to loop through arrays in Java?

- Use a **for** indexing from zero to the array length.
 - Or using a special **for** syntax, also called **for-each**
- **Wrong indexing can generate runtime errors.**

What are multidimensional arrays?

- Are powerful structures and can be useful for Matrix operations

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?

The next lecture will cover: File I/O and Collections